

RAPPORT TECHNIQUE CODEV

Editeur de Labyrinthe Pyrat

Encadrant : Bastien Padeloup

18 JUIN 2020

Groupe :

Avare Quentin

Le Gall Théo

Pageau Aurélien

1 Résumé

Dans le cadre du projet Conception et Développement de Système d'Ingénierie ou projet CODEV, les étudiants d'IMT Atlantique apprennent à concevoir et réaliser un projet technique répondant à un besoin initial précis. Les élèves doivent donc planifier, concevoir, analyser et développer une solution permettant de satisfaire ce besoin.

Notre projet « Editeur de Labyrinthe PyRat » répond à un besoin bien précis de l'enseignant encadrant en termes de pédagogie. Dans son UE « Algorithmique et Mathématiques discrètes » ou PyRat, les étudiants apprennent à concevoir, développer et améliorer une Intelligence Artificielle. Ce cours utilise comme support un jeu faisant intervenir un rat et un python mis en compétition dans un labyrinthe. Pour gagner, ils doivent récupérer plus de fromages que leur adversaire. Tout au long de cette UE, les élèves suivront une multitude d'ateliers à la pédagogie révolutionnaire et pourront ainsi développer leur IA et finalement la faire concourir dans un tournoi. Cet apprentissage faisant intervenir amusement et créativité sert avant tout à aborder plusieurs concepts complexes comme la théorie des graphes dans un labyrinthe.

Un autre support de cette UE est le site web du cours. Celui-ci sert avant tout de plateforme de cours en ligne ; les étudiants peuvent ainsi travailler chez eux avec la totalité des explications et exemples d'un cours standard au format papier. Cependant, ce cours en ligne ne permet pas de réaliser une des tâches les plus répétitives et importantes du cours PyRat : l'expérimentation.

« Un produit ne fonctionne pas s'il n'a jamais été testé »

Sans cette phase de test, le bon déroulement d'un projet ne peut être assuré.

Notre projet CODEV s'inscrit dans une démarche d'amélioration de l'UE PyRat. Notre solution exploitera les supports existants tout en permettant aux étudiants de mieux s'approprier cette phase de test en la rendant personnalisable et modulable.

Nous permettrons ainsi aux étudiants **d'améliorer leur compréhension du labyrinthe** tout en leur permettant de réaliser, de façon ludique, un outil de test pour leurs algorithmes

Table des matières

1	Résumé.....	2
2	Introduction	4
3	Objectifs du projet.....	5
3.1	Enjeux.....	5
3.2	Contraintes	5
3.3	Cahier des charges	6
3.4	Planning et répartition des taches	6
3.5	Fonctionnalités et contraintes	7
4	Développement de l'application web	8
4.1	Structure de l'application.....	8
4.2	IHM et fonctions d'affichage.....	11
4.3	Labyrinthe	13
4.4	Tests	16
5	Utilisation de l'application	19
5.1	Mode d'emploi	19
5.2	Cas d'utilisation	21
6	Conclusion	23
7	Références.....	23
8	Glossaire.....	24

2 Introduction

Dans le cadre de la formation FISE A1, les étudiants ont pu étudier de manière ludique la théorie des graphes et l’algorithmique en langage python. En effet, la formation permet aux étudiants de réaliser des programmes permettant de parcourir un labyrinthe. Ces programmes ont pour but de familiariser les élèves à la théorie des graphes et à l’intelligence artificielle.

Le labyrinthe, généré automatiquement n’est pas personnalisable et est difficile à manipuler. De plus, le code source à l’origine du labyrinthe est difficile à appréhender et donc à manipuler.

Dans ce livrable, vous trouverez les caractéristiques, les méthodes et la planification de notre réponse à ces problématiques.

L’organisation de ce rapport est le suivant :

- 1) La première partie (grand 3) explique les attentes du client vis-à-vis du projet. Il décrit ses objectifs et les mets en perspectives avec nos solutions.
- 2) La deuxième partie explique en détails la solution technique adoptée pour répondre au besoin du client. Cette description se fait par fonctionnalités et se termine par une validation technique du produit.
- 3) La troisième et dernière partie décrit les cas d’utilisations, où se trouve et comment utiliser chaque fonctionnalité.

3 Objectifs du projet

Cette partie a pour objectif de décrire les besoins du client et les fonctionnalités de notre solution.

3.1 Enjeux

Notre projet CODEV doit ainsi permettre aux responsables de l'UE PyRat d'améliorer leur pédagogie en permettant aux étudiants d'approfondir leur compréhension de la théorie des graphes de façon ludique.

Le projet devra ainsi :

- Permettre aux utilisateurs du site de générer un labyrinthe de type PyRat
- Rajouter et enlever des objets dans le labyrinthe (Murs, boue, fromages)
- Permettre le téléchargement du labyrinthe sous la forme d'un fichier utilisable par le programme python PyRat
- Permettre l'import sur le site d'un labyrinthe déjà réalisé

3.2 Contraintes

Le projet devra être ainsi accessible depuis le site de la formation PyRat. Il sera utilisé par les étudiants et les responsables de l'UE pour effectuer des tests et / ou développer de nouvelles fonctionnalités. La partie technique devra ainsi être modifiable facilement et résiliente.

D'un point de vue technique quand un utilisateur réalise un labyrinthe non valide à l'export (non-conformité avec les règles établis dans le jeu PyRat) une indication sera émise.

La longueur du labyrinthe devra être comprise entre 1 et 50 (de même pour la largeur).

Il n'y aura pas de limite au niveau du nombre de fromages et boues que l'utilisateur pourra mettre sur le labyrinthe.

Le projet devra être accessible depuis un onglet se trouvant sur le site du cours Pyrat

3.3 Cahier des charges

L'éditeur de labyrinthe prendra la forme d'un site internet accessible via le site PyRat. Un onglet sera effectivement disponible dans le menu du site pour y accéder. Au centre de ce nouveau site web on retrouvera notre labyrinthe, sous forme d'une grille dont la taille sera celle par défaut du programme PyRat.

L'utilisateur pourra utiliser deux inputs situés au-dessus de cette grille afin de redimensionner le labyrinthe en intervenant respectivement sur la longueur (X) et la largeur (Y). La grille sera par défaut de la même taille que celle utilisée en cours. Un bouton sera situé à côté afin de régler les variables X et Y en mode tournoi.

Afin maintenant de modifier ce labyrinthe, l'utilisateur pourra utiliser un menu sous forme de barre latérale à gauche de cette grille qui proposera trois objets : Fromage, Mur et Boue. Ces objets pourront être ainsi disposés à n'importe quel endroit du labyrinthe. Pour les déposer, il suffira de sélectionner l'un des objets puis de cliquer sur la zone ciblée. Si l'utilisateur souhaite annuler une modification, il lui suffira de cliquer sur la zone en question.

On retrouvera aussi deux boutons dans la partie supérieure du site web, l'un afin d'exporter le modèle de labyrinthe et le suivant afin d'importer un modèle déjà existant. Le tout dans un format compatible avec le programme PyRat.

Le projet a commencé le 10 mars 2020 et le livrable devra être remis au client le 25 juin 2020.

3.4 Planning et répartition des tâches

Etudiants	Taches	Outils
Théo LE GALL	Développement des principales fonctionnalités	HTML, CSS, JavaScript, PHP, Python
Quentin AVARE	Organisation, relation client, aide à la programmation	HTML, CSS, JavaScript
Aurélien PAGEAU	Rédaction	

Etapes	28/02 – 10/03	10/03 – 26/05	26/05 – 18/06	18/06 – 26/06
Analyse du besoin et familiarisation avec les outils				
Développement de toutes les fonctionnalités				
Tests et développement de la partie export/import				
Finalisation				

3.5 Fonctionnalités et contraintes

En résumé voici les fonctionnalités de notre application web :

FP1 : Permettre la modification des dimensions d'un labyrinthe PyRat en ligne

FP2 : Permettre le rajout de fromages, boues, murs sur un labyrinthe

FP3 : Importer n'importe quel type de labyrinthe dans l'application web

FP4 : Exporter le labyrinthe modifié depuis le site PyRat

Voici les fonctions de contraintes permettant d'assurer la validation du besoin du client :

FC1 : L'application est accessible depuis le site PyRat

FC2 : Indiquer à l'utilisateur si le labyrinthe est conforme

FC3 : Les dimensions du labyrinthe sont comprises entre 0 et 50 cases

FC4 : Aucunes limites sur la quantité de fromages pouvant être sur le labyrinthe

4 Développement de l'application web

4.1 Structure de l'application

Avant de créer l'application, nous avons dû nous positionner par rapport à cette dernière, quels langages utiliser (partir sur des langages de programmation web basiques ou alors implémenter l'application grâce à Unity, réputé pour le développement de jeux vidéo), quel type de programmation web (frontend et/ou backend) ... En effet, le projet étant conséquent par rapport à ce dont nous avons affaire d'habitude, il était important de se positionner et de prendre du recul avant de rentrer dans la phase de code.

Nous avons alors opté pour une programmation frontend. Effectivement, cette programmation tournée vers le client a pour but d'une part de gérer l'affichage de la page web mais aussi de traiter les interactions du client avec la page web. Le backend quant à lui, s'occupe du côté serveur, à savoir l'hébergement, le traitement des données etc. Comme énoncé dans le cahier de charge, l'application doit présenter un système d'import-export de fichier, ce qui semble plus adapté au backend, mais nous verrons par la suite que ce dispositif peut très bien être géré en frontend dans certains cas précis.

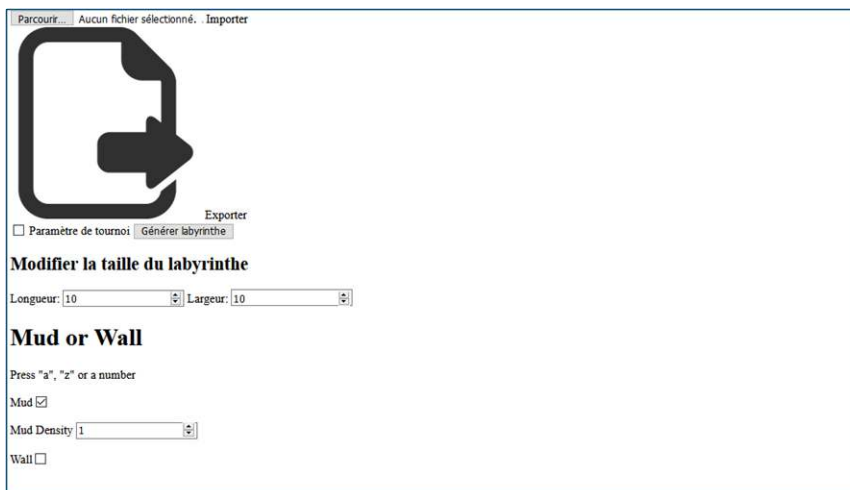
En frontend, nous retrouvons 3 langages de programmation principaux : HTML, CSS et JavaScript. Ce sont effectivement ces 3 langages dont nous nous sommes servis. En plus de ces langages, il existe différents Framework pour ajouter des fonctionnalités plus poussées, parmi les plus connus nous pourrions citer : JQuery, Angular, Bootstrap, Foundation, Semantic UI. Nous avons fait le choix de nous passer de ces outils, tout d'abord car nous débutons en développement web, s'encombrer d'outil supplémentaire allait être contreproductif, mais aussi pour rendre le code le plus simple possible pour que ce dernier puisse être compris de tous. Penchons-nous désormais sur les 3 langages prisés du développeur frontend.

Tout d'abord, étudions l'HTML (HyperText Markup Language) qui actuellement est à la version 5 (noté HTML5). Il s'agit d'un langage de balisage. En effet, la structure du code est sous forme de « balise », des éléments qui suivent la syntaxe suivante :

« <>Balise</> ». C'est un langage simple et intuitif qui permet de structurer sémantiquement la page web, de gérer, grâce à ces balises, la forme du contenu. Voici un exemple de notre code html :

```
<!-- Box bouton largeur longueur-->
<div id="mazeSize" class="intoBar">
  <h2>Modifier la taille du labyrinthe</h2>
  <label for="widthValue">Longueur:</label>
  <input id="widthValue" type="number" value="10" min="5" max="50" />
  <label for="heightValue">Largeur:</label>
  <input id="heightValue" type="number" value="10" min="5" max="50" />
</div>
```

HTML5 nous a alors permis d'afficher du texte (balise h2 qui signifie Titre2 comme un style dans Word), des inputs (boutons, boîtes de texte etc.) On remarque aussi dans certaines balises les éléments « id » et « class ». Ces deux éléments nous permettront d'ajouter et de modifier les fonctionnalités des balises html grâce aux autres langages. Id permet d'identifier une balise en particulier (l'id est ainsi unique) quand class nous permet de modifier toutes les balises ayant la même class. Voici donc notre application web une fois le code HTML terminé : (il est toutefois important de préciser que nous n'avons pas codé entièrement chaque langage l'un à la suite de l'autre, nous avons progressivement complété chacun d'entre eux)



Parcourir... Aucun fichier sélectionné. Importer

Exporter

☐ Paramètre de tournoi

Modifier la taille du labyrinthe

Longueur: Largeur:

Mud or Wall

Press "a", "z" or a number

Mud ☒

Mud Density

Wall ☐

L'application web prend forme mais soyons honnête, elle est peu attirante. C'est là qu'intervient notre second langage : le CSS)

Le CSS pour Cascading Style Sheets est très souvent associé à l'HTML. En effet, c'est ce langage qui va gérer la présentation de l'application web. On peut ainsi modifier les couleurs, le style d'écriture (police, taille, gras, italique...) mais aussi et surtout la disposition. On peut voir sur l'image précédente que tous les éléments se suivent de haut en bas, la largeur de la page n'est pas du tout exploitée. En théorie, l'application

reste faisable sans CSS, mais nous verrons par la suite que cela facilite grandement l'implémentation du labyrinthe. Voici un exemple de notre code CSS :

```
#container {
  position: center;
  display: grid;
  grid-gap: 0px;
  grid-template-columns: repeat(var(--grid-cols), 50px 20px) 50px;
  grid-template-rows: repeat(var(--grid-rows), 50px 20px) 50px;
  border: solid 5px;
}

.grid-item {
  text-align: center;
}
```

Nous avons précédemment énoncé le principe d'id et de class en HTML, voici l'une de leur utilité. Le # permet de modifier une balise par son id, on va donc modifier une unique balise (ici la balise avec id = « container »). Et donc le « . » va nous permettre de modifier toutes les balises ayant la même class. Une nouvelle fois, on peut voir qu'il s'agit d'un langage relativement simple à appréhender. Voilà notre application une fois le code CSS ajouté :


Importer


Exporter

■

Paramètre de tournoi

Générer labyrinthe

Modifier la taille du labyrinthe

Longueur: 10

Largeur: 10

Mud or Wall

Press "a", "z" or a number

Mud ☒

Mud Density 1

Wall ☐

■

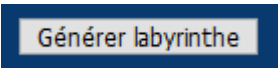
Le site est déjà plus organisé et visuellement plus agréable. Toutefois, ce site ne fait actuellement rien. Effectivement, interagir avec les différents inputs ne produira rien, il faut pour cela utiliser le dernier langage.

Le langage JavaScript est ainsi le langage qui va nous permettre d'implémenter des actions à toutes nos balises. Le JavaScript permet d'écrire des scripts pour les pages web, tout comme la Java il s'agit d'un langage orienté objet. Il serait difficile de vous présenter ce langage à la manière des précédents car celui-ci est plus complet et donc

complexe. En effet, il se rapproche plus des langages dont nous avons l'habitude comme le python, le Java, le C. On utilise des variables, créer des fonctions etc. C'est dans ce langage que réside tout notre projet. Et une nouvelle fois, pour ajouter des fonctionnalités à nos balises HTML, nous allons utiliser leur id et class.

```
<input id="generateMaze" type="button" value="Générer labyrinthe">
```

(Le code HTML)



(Son rendu sur l'application)

```
var generateMaze = document.getElementById("generateMaze");
```

(Le code JavaScript)

On peut grâce à la fonction « `getElementById()` » utiliser notre bouton comme un objet JavaScript à partir de l'id de ce dernier. On peut ensuite associer à cet objet un évènement, une action au clique. Etudions le code suivant :

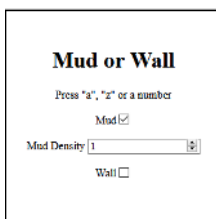
```
generateMaze.addEventListener('click', (event) => {  
  //On empeche la modification de longueur largeur:  
  mazeHeightButton.disabled = true;  
  mazeWidthButton.disabled = true;  
  tournamentMod.disabled = true;  
  //Appel de makeRows  
  makeRows (mazeWidthButton.value*2-1,mazeHeightButton.value*2-1);  
})
```

On utilise la fonction « `addEventListener(type,event)` » où « `click` » désigne trivialement que l'action s'effectue au clique et « `event` » va être l'action à exécuter. C'est donc sur le principe d'évènement associé à une balise que nous avons réalisé notre projet.

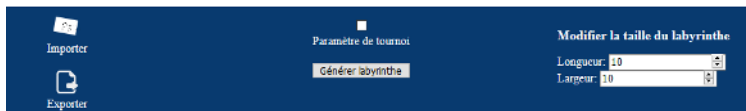
Nous voilà désormais avec 3 outils que sont les langages du développeur frontend. Penchons-nous maintenant sur les différents éléments de notre application.

4.2 IHM et fonctions d'affichage

On peut séparer l'application web en trois parties. Une partie centrale, une partie gauche et une partie haute :

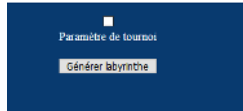


Partie Gauche



Partie Haute

La partie centrale est actuellement vide, c'est celle qui servira à accueillir le labyrinthe. La partie haute peut être divisée en trois sous partie : gauche, centre et droite.

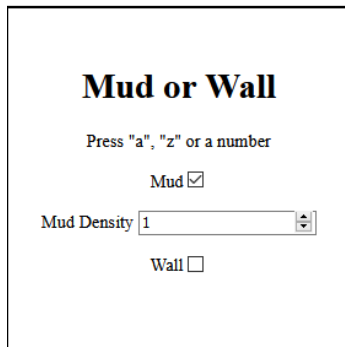


La partie droite contient deux inputs de type number (en HTML), ce sont simplement des boîtes dans lesquels nous pouvons ajouter des valeurs numériques (et seulement numérique). Ces valeurs nous serviront à modifier la taille du labyrinthe avant de générer ce dernier. Elles sont simplement stockées dans des variables (dans le code JavaScript). Pour cela nous avons utilisé la fonction vu à la partie précédente : `getElementById()` à laquelle nous avons ajouté « `.value` » nous permettant d'obtenir la valeur indiquée. Par défaut, cette valeur est à 10 pour la longueur et la largeur.

La partie centrale est composée de deux différents inputs. Le premier est affiché comme un petit carré, c'est ce qu'on appelle en HTML une `checkbox`. Cette `checkbox` n'a que deux états : coché et décoché. Comme l'indique le texte du dessous, cet input permet d'associer les valeurs de longueur et de largeur aux valeurs par défaut en mode tournoi. Une nouvelle fois, dans le code JavaScript on utilise `getElementById()` avec ici « `.checked` » qui renvoie un booléen : `TRUE` si l'input est à l'état coché et `FALSE` le cas échéant. Grâce à des conditions, on peut changer la valeur de nos inputs longueur, largeur. Le second élément est un simple bouton : « Générer labyrinthe » qui comme son nom l'indique permet de générer le labyrinthe. Mais nous nous attarderons sur cet élément dans la partie suivante.

Dernière partie de notre bandeau supérieur : Les deux boutons import, export. Ces deux éléments permettent effectivement l'import et l'export du labyrinthe, leur fonctionnement est un peu plus complexe que les précédents. L'export permet d'exporter le labyrinthe sous forme de fichier texte. Habituellement, ce type de manœuvre se fait côté serveur, ici le fichier peut être assimilé à une simple variable `String` que l'on peut exporter. On pouvait aussi simplement copier la variable dans le presse-papier du client mais ce n'est pas compatible avec tous les navigateurs. De même, l'import permet de déposer un fichier texte sur le site mais ce fichier sera directement intégré par une variable `String`, inutile donc de passer côté serveur.

Connaissant désormais le fonctionnement du bandeau supérieur, attardons-nous désormais sur celui de gauche :

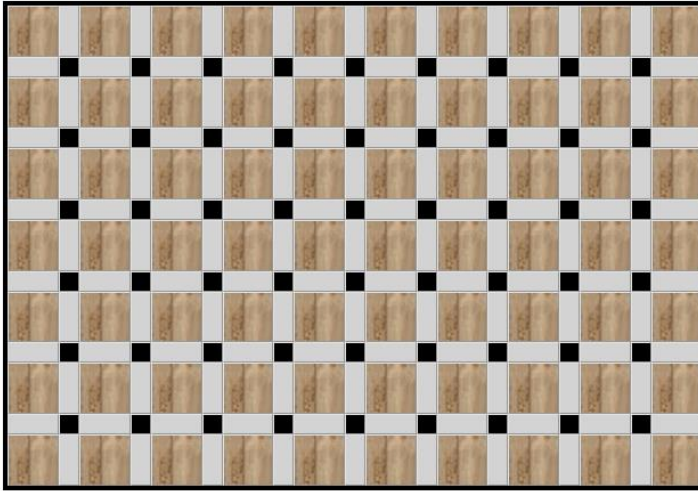
The screenshot shows a window titled "Mud or Wall". Inside, there is a text prompt "Press 'a', 'z' or a number". Below this, there are two checkboxes: "Mud" which is checked with a small square icon, and "Wall" which is unchecked. Between these checkboxes is a label "Mud Density" followed by a text input field containing the number "1" and a small spinner control on the right side of the field.

Il dispose d'un titre : Mud or Wall. En effet, il s'agit d'un menu qui permet d'interagir avec le labyrinthe. On retrouve deux checkBox Mud et Wall qui permettent de déposer sur le labyrinthe Mur ou Boue. Nous étudierons ceci plus en profondeur avec le labyrinthe. On a aussi un input pour modifier la densité de la boue. Celle-ci fonctionne exactement comme les inputs longueur et largeur : on récupère avec `getElementById()` et on met la valeur (`.value`) dans une variable. Petite particularité toutefois, le menu propose d'appuyer sur a, z ou un nombre. Ainsi, l'application est capable de capter les frappes clavier. Ce qui permet d'activer la boue en appuyant sur « a » ou les murs en appuyant sur « z », les nombres eux permettent de modifier la densité de la boue. Ceci a été réalisé dans un but ergonomique, les labyrinthes pouvant contenir de nombreuses cases, il semblait important de pouvoir alterner rapidement d'un mode à l'autre.

Il reste ainsi une autre partie du site à étudier, il s'agit de la partie associée au labyrinthe. Cette partie étant plus complexe et essentielle à la compréhension du projet, nous allons l'étudier dans la partie suivante.

4.3 Labyrinthe

Le but de notre projet rappelons-le, est d'éditer en ligne un labyrinthe. Toute la partie IHM vu précédemment n'a donc qu'un seul but : interagir avec ce labyrinthe. Regardons tout d'abord le rendu de ce dernier avant d'étudier plus en détail son fonctionnement.



Le voici : nous avons pris ici une longueur de 7 et une largeur de 10. Il se présente sous la forme d'une grille exclusivement composée de boutons. En effet, chaque éléments, cases, intersections, sont en fait des boutons. D'un point de vue ergonomique, il est donc facile de modifier ce labyrinthe. On peut modifier ses cases et intersections simplement en cliquant.

Tout commence par le bouton « Générer labyrinthe ». En effet, dans le code JavaScript, le bouton une fois cliqué va activer la fonction suivante :

```
makeRows (mazeWidthButton.value*2-1,mazeHeightButton.value*2-1);
```

On remarque que les valeurs de longueur et largeur (« mazeWidth/HeightButton.value ») ont été modifiées. En effet, ces coordonnées correspondent au nombre de case du labyrinthe, mais ici la fonction va générer aussi les intersections. Un labyrinthe de $n*m$ cases va donner sur l'application un total de $(n*2-1)*(m*2-1)$ cases et intersections. Pour ensuite représenter ce labyrinthe, nous allons utiliser le code CSS, c'est cette partie qui fait que nous ne pouvons nous passer du CSS.

```
position: center;
display: grid;
grid-gap: 0px;
grid-template-columns: repeat(var(--grid-cols), 50px 20px) 50px;
grid-template-rows: repeat(var(--grid-rows), 50px 20px) 50px;
border: solid 5px;
```

On utilise le display « grid ». Un display en CSS permet de gérer l'alignement des objets HTML, grid permet ici d'aligner les objets sous forme de grille. Le nombre de ligne et de colonnes de la grille est géré par « grid-template-rows/columns ». On utilise la fonction « repeat(nombre, élément) » qui permet de répéter 'nombre' de fois 'élément'. Les valeurs de 'nombre' sont stockées dans des variables données par le code JavaScript :

```
container.style.setProperty('--grid-rows', Math.floor(rows/2));
container.style.setProperty('--grid-cols', Math.floor(cols/2));
```

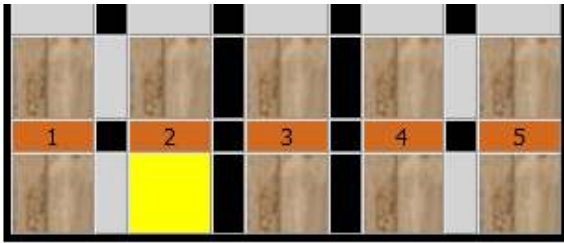
Comme on répète case de taille 50px (pixels) puis case de taille 20px, il faut diviser par deux (et garder la partie entière) nos coordonnées longueur largeur. Mais en faisant ça, il nous reste les dernières cases en longueur et en largeur qui n'ont pas de tailles assignées, c'est pour cela qu'on trouve un 50px après le « repeat ». Nous nous retrouvons donc avec un labyrinthe sous forme de grille affiché à l'écran.

Pour donner vie à ces nombreux boutons, toujours dans la fonction makeRows(), nous allons parcourir toutes la grille (une fonction for est disponible sur JavaScript) de gauche à droite en descendant à la fin de chaque ligne. Et nous allons pouvoir assigner chaque bouton à une class (celles de l'HTML). On utilisera Objet.className qui permet d'assigner une class à un objet. Et donc pendant le parcours, pour les lignes impaires, une case sur deux sera assignée à la class « intersection » (l'autre restera une « case »). Pour les lignes paires, une case sur deux sera une « intersection », l'autre sera une « case morte » (ces cases sont inutiles, elles n'ont pas besoin d'être manipulées). De plus, pendant le parcours, nous avons changé l'« id » des cases (id du langage HTML) en « x_y » avec « x » et « y » les coordonnées en longueur et en largeur. On distingue ces nouveaux id sur l'image suivante (on a affiché seulement les « id » des cases mais les « intersections » et « cases mortes » en ont aussi).

0_0	0_2	0_4	0_6	0_8	0_10	0_12	0_14	0_16	0_18
2_0	2_2	2_4	2_6	2_8	2_10	2_12	2_14	2_16	2_18
4_0	4_2	4_4	4_6	4_8	4_10	4_12	4_14	4_16	4_18
6_0	6_2	6_4	6_6	6_8	6_10	6_12	6_14	6_16	6_18
8_0	8_2	8_4	8_6	8_8	8_10	8_12	8_14	8_16	8_18
10_0	10_2	10_4	10_6	10_8	10_10	10_12	10_14	10_16	10_18
12_0	12_2	12_4	12_6	12_8	12_10	12_12	12_14	12_16	12_18

Pour faire le point, nous avons un labyrinthe affiché, les cases sont séparées sous différentes catégories (intersection, case et case morte). Et nous pouvons les différencier entre elles par leur id. Un peu de CSS et nous avons des cases de couleurs différentes.

Désormais, nous souhaitons pouvoir éditer ce labyrinthe. Comme cases et intersections sont des boutons, il reste simplement à regarder quelle checkBox (Boue ou Mur) est validée et modifier le bouton en influant sur le code CSS du bouton en question :

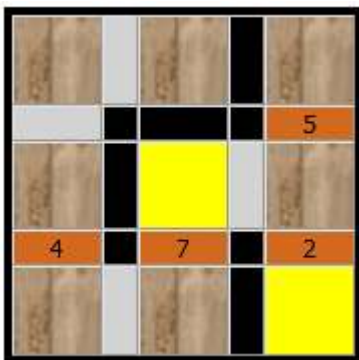


Une case cliquée se changera en jaune pour traduire la présence d'un fromage. Une case en noire correspondra à un mur et une case marron/orange à de la boue. Petite spécificité, on affiche la densité de la boue. Mais comment mémoriser ce choix notamment pour

l'exporter ? Actuellement, la case est juste colorée, rien n'est enregistré.

En effet, il reste une action que doit réaliser la fonction `makeRows()`. La fonction va créer une matrice (au départ nulle) de taille $(n*2-1)*(m*2-1)$, cela revient à modéliser tout le labyrinthe sous forme de matrice. Comme chaque bouton de la grille dispose d'un id qui donne ses coordonnées, on peut modifier la valeur à ces mêmes coordonnées dans la matrice. Mettre un mur sur une intersection mettra la valeur -1, une case vide donnera 0 et pour la boue, on mettra la valeur de la densité.

Typiquement 2 de densité donnera 2 sur la matrice. Un fromage mettra la valeur 1 (aucune confrontation avec la boue de densité 1 car elles n'ont pas la même place dans la matrice). Tout est désormais en mémoire, il n'y a plus qu'à changer la matrice au format compatible avec PyRat. Un simple parcours de la matrice permet de passer de l'un à l'autre.



```
[[0, 0, 0, -1, 0],
 [0, 0, -1, 0, "5"],
 [0, -1, 1, 0, 0],
 ["4", 0, "7", 0, "2"],
 [0, 0, 0, -1, 1]]
```

```
B
3
5 0 0 1
8 0 1 0
3 0 0 0
1 5 0 0
0 8 0 1
6 3 1 0
0 1 0 1
0 0 1 0
0 6 0 0
0
8
2 4
```

4.4 Tests

Les différentes fonctionnalités du site sont maintenant codées, il faut maintenant les mettre à l'épreuve. En effet, l'application ayant pour but d'être ajoutée au site PyRat, il est important qu'elle puisse rester fonctionnelle. Voici la liste de test que nous avons suivi pour valider notre projet :

⇒ Générer le labyrinthe

-Générer un labyrinthe alors qu'un est déjà présent

Pour éviter cette situation nous avons tout simplement désactivé le bouton « Générer labyrinthe » une fois que celui-ci ait été activé une première fois

-Mettre des valeurs problématiques en longueur largeur : trop grandes puis négatives

D'une part, l'input bloque les valeurs en dehors de l'intervalle [1,50] mais ceci ne marche que si l'utilisateur modifie le nombre avec les flèches. Entrer un nombre manuellement ne respectera pas cette règle. Alors nous avons décidé au sein de la fonction makeRows() d'afficher un message d'erreur si les valeurs entrées sont négatives ou supérieure à 99 ($50*2-1$) et ainsi de sortir de la fonction sans avoir générer de labyrinthe.

-Activer le mode tournoi et modifier les valeurs de longueur largeur

Pour pallier à ce problème, nous avons choisi de désactiver les input longueur largeur lorsque le mode tournoi est activé. De plus, désactiver ce mode va permettre d'utiliser de nouveau les inputs longueur largeur.

-Modifier la longueur largeur une fois le labyrinthe généré (et activer mode tournoi)

Nous avons utilisé la même astuce : désactiver les inputs longueur largeur et mode tournoi une fois que le labyrinthe ait été généré. Donc une fois généré, la taille du labyrinthe ne pourra plus être modifiée. Pour en générer un nouveau, il faudra actualiser la page.

⇒ Modifier le labyrinthe

-Alterner entre mode boue et mur

Les deux inputs ne peuvent être activé en même temps. En effet, cocher la case boue va immédiatement désactiver la case Mur et la décocher va réactiver cette case. De même, cocher la case Mur va désactiver la boue et la décocher la réactivera. On ne peut donc avoir à la fois mur et boue de sélectionnés. De plus, l'un de ces modes est toujours activé. On ne peut pas ainsi cliquer sur une case sans que boue ou mur ne soit sélectionné.

-Mettre une densité problématique (trop grande puis trop petite)

Comme pour les inputs longueur largeur, nous avons imposé une limite sur l'input. Encore une fois, manuellement l'utilisateur peut entrer une valeur problématique. Alors, si ce dernier insert une valeur négative, elle sera mise à la valeur minimale dans

le labyrinthe (Mettre une densité de -2 donnera une densité de 1). Même principe si l'on met une valeur supérieure à 9, celle-ci sera arrondi à 9.

-Modifier un élément du labyrinthe déjà placé.

Si par exemple un mur est déjà placé mais nous souhaitons finalement mettre de la boue. Cliquer une nouvelle fois sur l'intersection en question la remettra à vide (même s'il s'agit d'un mur et que l'outil sélectionné est la boue). Et une fois à vide, cliquer dessus avec la boue ou un mur de sélectionné déposera l'élément en question. De plus, la matrice qui sert à stocker ces informations sera elle aussi mise à jour. Même principe pour les fromages, cliquer sur un fromage le fera disparaître.

-Créer un labyrinthe non connexe (zone non explorable)

Actuellement, ce problème n'est pas pris en charge. Nous nous assurerons que l'utilisateur reçoive un message d'erreur s'il tente d'exporter un labyrinthe non connexe. On utilisera pour cela les valeurs propres de la matrice laplacienne associée au graphe.

-Limite de fromage

Nous avons souhaité laisser une liberté totale à l'utilisateur concernant le nombre de fromages.

⇒ **Export et Import**

-Tester si le fichier exporter correspond bien au labyrinthe notamment avec des labyrinthes exotiques (vide ou rempli de murs)

Peu importe le labyrinthe, le fichier exporté correspond bien. Le jeu PyRat exécute tous ces labyrinthes.

-Tester si l'import correspond bien au labyrinthe en question

L'import fonctionnant sur les mêmes bases que l'export, il n'y a pas non plus de problèmes et ce peu importe le labyrinthe

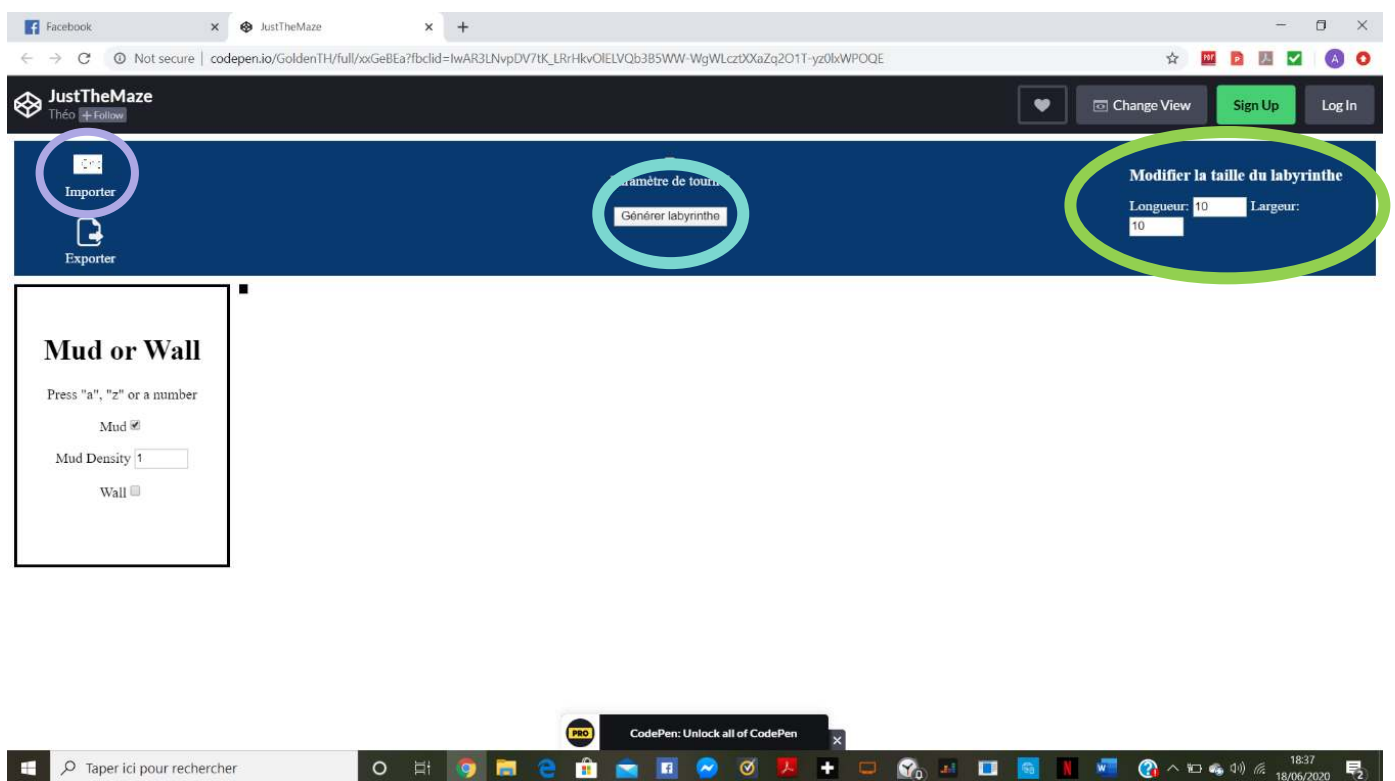
-Tester un fichier autre qu'un fichier maze

Actuellement, l'application ne gère pas ce problème. Nous pourrions par la suite ajouter un message d'erreur si le fichier n'arrive pas à être lu et ainsi faire en sorte que l'utilisateur puisse réessayer d'importer un fichier sans actualiser la page.

5 Utilisation de l'application

5.1 Mode d'emploi

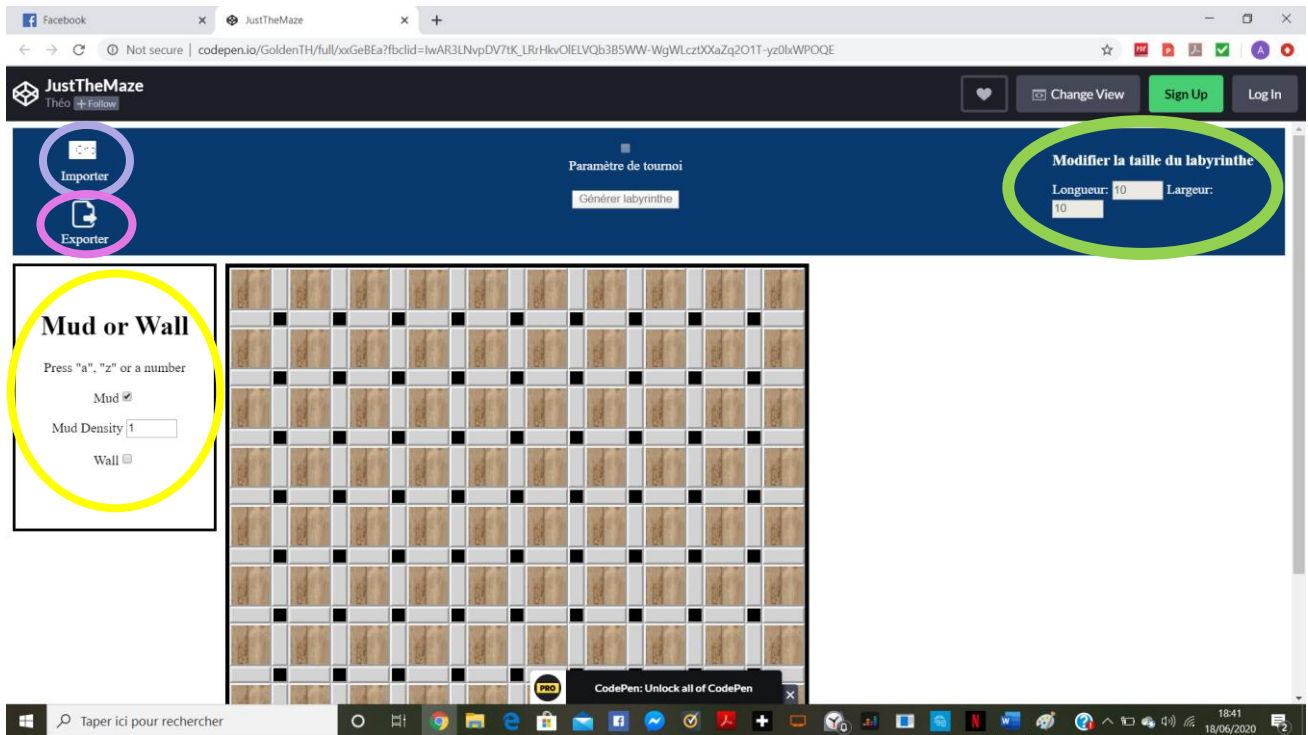
L'application a été développée dans l'optique de faciliter et d'améliorer le déroulement de l'UE PyRAT. C'est pourquoi son intégration au site web déjà existant est très importante. Tout d'abord il faut accéder au site PyRAT disponible sur Moodle IMT Atlantique. Il faudra ensuite se rendre dans l'onglet correspondant à notre application. Ensuite une page similaire à celle ci-après va apparaître :



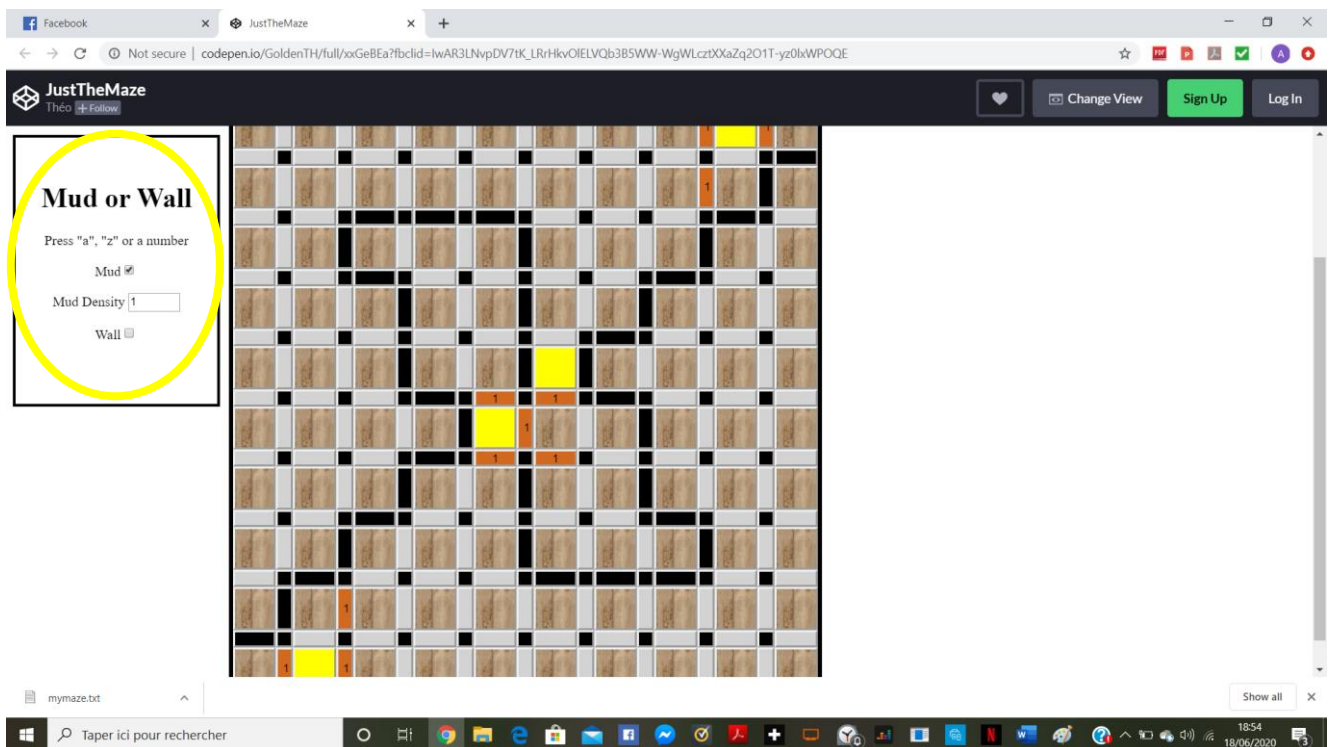
L'utilisateur va alors pouvoir générer un labyrinthe vide en précisant ses dimensions grâce à l'interface et au bouton « générer un labyrinthe ».

Le bouton importation permet d'importer un fichier .maze décrivant le layout d'un labyrinthe déjà existant. Une fenêtre permettant de modifier le labyrinthe importé s'ouvrira par la suite. Les boutons sur le bandeau supérieur à droite permettent de modifier la taille du labyrinthe (avant de le générer).

Dans tous les cas la fenêtre suivante s'ouvre alors :



Une fois le labyrinthe créé, le client va pouvoir cliquer sur les cases marrons pour faire apparaître les fromages jaunes et cliquer sur les barres grises en sélectionnant « mud » pour de la boue et « wall » pour un mur. Une fois la création terminée, on obtient un labyrinthe qui ressemble à :

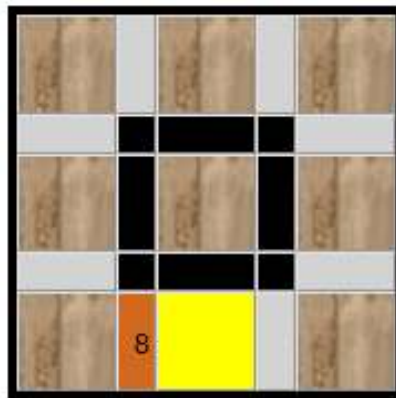


Une fois la modification du labyrinthe terminée, l'utilisateur pourra l'exporter et l'enregistrer sur son système grâce au bouton « export » situé en haut à gauche de l'écran.

5.2 Cas d'utilisation

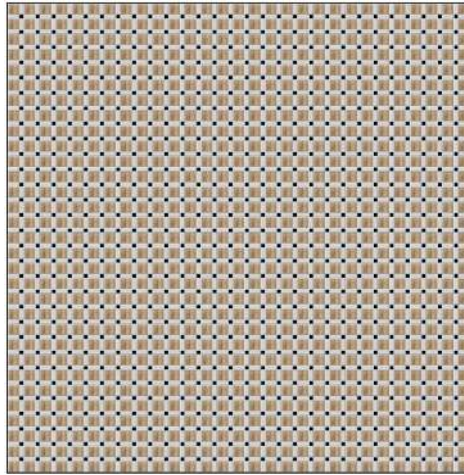
1- Premier cas : Test d'un algorithme Pyrat

Pour tester un algorithme spécifique Pyrat (du type greedy ou BFS), il suffit d'importer ou de créer un labyrinthe et de modifier les cases (murs, boues, fromages) de façon à réaliser l'architecture désirée (voir exemple). L'utilisateur va choisir ici de générer un labyrinthe de longueur et de largeur 3 afin de réaliser une expérience simple (inutile de s'encombrer de trop de cases). Il va sélectionner l'outil boue et modifier sa densité à 8 : nombre supérieur au nombre de mouvement nécessaire pour faire le tour sans passer par la boue. Et il va ensuite sélectionner l'outil mur pour faire la deuxième issue vers le fromage, pour tester son algorithme. Il pourra finalement exporter ce labyrinthe en cliquant sur le bouton. L'importer sur le jeu est désormais de l'ordre de l'application Pyrat.



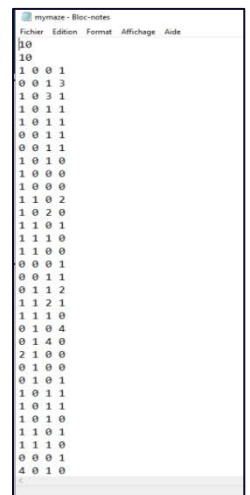
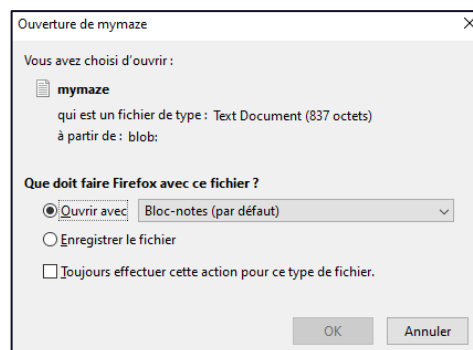
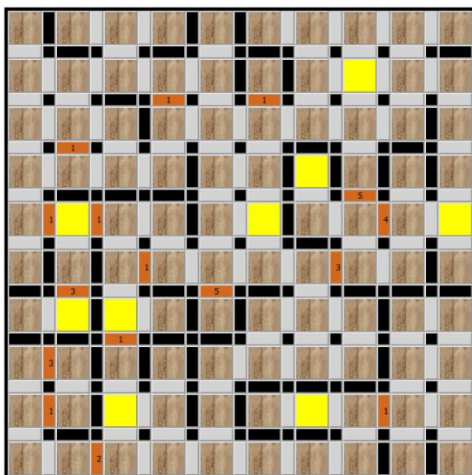
2- Deuxième cas : Création d'une carte de tournois

Notre utilisateur souhaite désormais mettre à l'épreuve son algorithme pour le tournoi. Pour créer une carte de tournois il suffit de cocher la case paramètres de tournois, qui modifie les valeurs de longueur et de largeur pour les valeurs de tournoi par défaut. Il cliquera ensuite sur le bouton générer tournoi. Puis il pourra disposer chaque objet de façon symétrique sur celui-ci. Et finalement, il cliquera sur le bouton exporter.



3- Troisième cas : Customisation d'un labyrinthe existant

Pour créer une carte custom, il suffit de choisir la taille en haut à droite de l'éditeur puis de disposer selon ses préférences chaque objet (fromage, murs, boues) sur le labyrinthe. Attention ! Si le labyrinthe a une partie fermée, un message d'erreur s'affichera. Si il n'y a aucun problèmes, vous pourrez exporter le labyrinthe sous format .maze



4- Quatrième cas : Import d'un modèle réaliser au préalable

L'utilisateur s'est rendu compte qu'il a oublié de mettre son fromage sur le premier cas d'utilisation. Il se rend alors sur l'éditeur et sélectionne le bouton importer. S'ouvre alors une fenêtre pour sélectionner le fichier en question. Il sélectionne son fichier,

clique sur ouvrir. L'éditeur va alors régler les valeurs de longueur largeur des inputs et afficher le labyrinthe en question qui est de nouveau modifiable via le site. Il cliquera sur la case pour y ajouter son fromage et exportera le modèle.

6 Conclusion

Pour conclure, créer un site web interactif a été une véritable épreuve que nous estimons avoir surmontée et ce malgré certaines complications et l'obligation d'acquérir des compétences en un temps limité. En dépit de ces difficultés, nous sommes parvenus à concevoir, réaliser et développer un éditeur qui respecte en tout points le cahier des charges définit avec le client.

Au début du projet, un certain flou technique nous empêchait d'avancer sur les parties importantes. Mais après plusieurs échecs et mauvais choix nous avons finalement choisi les langages JavaScript, HTML et CSS sur CodePen, une plateforme de développement en ligne parfaitement adaptée à ce type de projet. Par la suite, il a été relativement simple de développer chaque fonctionnalité et de terminer le projet.

Ce projet complet et d'une réelle utilité, nous a donné une image du vrai monde du travail où un cahier des charges et un planning doivent toujours être bien conçus avant la phase de développement pour conduire un travail rigoureux, notamment sur une longue durée.

Quelques modifications sont encore à réaliser avant l'échéance finale. Nous souhaitons en effet rajouter certaines fonctionnalités et rendre la page web plus dynamique. Mais aussi et surtout d'implémenter cette dernière sur WordPress.

7 Références

WIKIPEDIA HTML disponible sur :

https://fr.wikipedia.org/wiki/Hypertext_Markup_Language (consulté le 10/02/20)

WIKIPEDIA CSS disponible sur :

[https://fr.wikipedia.org/wiki/Feuilles de style en cascade](https://fr.wikipedia.org/wiki/Feuilles_de_style_en_cascade) (consulté le 12/02/20)

WIKIPEDIA Javascript disponible sur :

<https://fr.wikipedia.org/wiki/JavaScript> (consulté le 14/02/20)

OpenClassrooms HTML&CSS disponible sur :

<https://openclassrooms.com/fr/courses/1603881-apprenez-a-creer-votre-site-web-avec-html5-et-css3> (consulté le 12/02/20)

OpenClassrooms JavaScript disponible sur :

<https://openclassrooms.com/fr/courses/2984401-apprenez-a-coder-avec-javascript> (consulté le 22/02/20)

Mozilla Feuille de style en cascade disponible sur :

<https://developer.mozilla.org/fr/docs/Web/CSS> (consulté régulièrement)

StackOverflow Forum en général disponible sur :

<https://stackoverflow.com/> (consulté très régulièrement)

WIKIPEDIA Théorie Spectrale des graphes disponible sur :

[https://fr.wikipedia.org/wiki/Théorie spectrale des graphes](https://fr.wikipedia.org/wiki/Théorie_spectrale_des_graphes) (consulté le 23/05/20)

8 Glossaire

Front-end : Développement de l'interface client. C'est-à-dire la partie visuelle du site web sur notre navigateur. Mais aussi les interactions, les animations et le responsive design.

Back-end : Développement de la partie serveur de l'application. Gestion du serveur, de l'hébergement et de la base de données.

Matrice Laplacienne : Matrice associée à un graphe G non orienté définie par $M = M_d - M_a$ (avec M_d la matrice des degrés de G et M_a la matrice d'adjacence de G)

Matrice des degrés : Matrice diagonale contenant des informations sur les différents degrés de chaque sommet

Matrice d'adjacence : Matrice de dimension $n \times n$ dont l'élément non diagonal a_{ij} est le nombre d'arêtes liant le sommet i au sommet j .

Unity : Moteur de jeu multiplateforme. L'un des plus répandus dans l'industrie du jeu vidéo.

Développement web : Le développement web repose sur l'utilisation des langages (HTML/CSS, JavaScript, PHP...) pour écrire des programmes qui sont ensuite exécutés par les ordinateurs. Les instructions sont mises en place sur Internet et sont effectuées sur des serveurs

Mud Density : Poids associé au chemin entre deux cases adjacentes. C'est-à-dire le nombre de déplacements nécessaires pour aller à une case voisine : vaut 1 sans boue, on ajoute la densité à cette valeur (2 de densité nécessitera donc 3 mouvements)